SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I Karanam Bhaskar Gnanesh(192324080) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Karanam Bhaskar Gnanesh
Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Scenario 1-Based Answer: AI-Powered Drone for Emergency Medicine Delivery

i. Greedy Best-First Search (GBFS) in this Scenario

Greedy Best-First Search selects the next node based only on the **heuristic value h(n)** (estimated straight-line distance to the destination). It ignores the actual cost already traveled.

Behavior:

- The drone always moves toward the location that appears closest to the destination.
- It makes quick routing decisions using the heuristic.
- If a path becomes blocked or weather increases travel cost, GBFS may still choose that direction because it does not consider the actual path cost.

Strengths:

- Very fast decision-making.
- Requires less computation.
- Suitable when immediate response is more important than finding the best route.

Weaknesses:

- Does not guarantee the shortest or safest path.
- Can get stuck exploring blocked or expensive routes.
- Performs poorly in highly dynamic and uncertain environments.
- May require frequent replanning when conditions change.

---

ii. A Search in this Scenario*

A* Search uses both:

- **g(n):** Actual cost from the starting point.
- **h(n):** Estimated cost (heuristic) to the goal.

It calculates:

f(n) = g(n) + h(n)

Behavior:

- The drone considers both the distance already traveled and the estimated remaining distance.
- It avoids routes with high travel cost caused by bad weather or difficult terrain.
- When real-time updates indicate blocked paths, A* recalculates a better route using the updated costs.

Advantages:

- Finds the optimal path if the heuristic is admissible.
- Balances speed and path quality.
- Produces safer and more reliable routes.
- Better suited for emergency delivery where both speed and safety matter.

---

iii. Impact of Heuristic Accuracy

Greedy Best-First Search

- **Accurate heuristic:** Quickly reaches the goal.
- **Inaccurate heuristic:** May choose wrong directions, causing delays and unnecessary exploration because it relies only on h(n).

A Search*

- **Accurate heuristic:** Finds the optimal path efficiently with fewer node expansions.
- **Less accurate heuristic:** Still finds the optimal path but explores more nodes, increasing computation time.

Conclusion:

- GBFS is highly dependent on heuristic accuracy.
- A* is more robust because it combines heuristic information with actual path cost.

---

iv. Effect of Dynamic Changes (Blocked Paths)

In a flood-affected region:

- Roads may suddenly become blocked.
- Weather conditions may change travel costs.

Greedy Best-First Search

- May continue moving toward blocked areas due to its heuristic.
- Often needs complete replanning.
- Performance decreases significantly in dynamic environments.

***A Search**

- Updates path costs using new information.
- Recalculates an alternative optimal route.
- Adapts better to changing environments, although replanning increases computation time.

---

v. Recommended Algorithm

*Recommended Algorithm: A Search**

Justification:

- Considers both actual travel cost and estimated remaining distance.
- Finds the optimal and safest route.
- Adapts better to changing conditions such as blocked roads and weather.
- Reduces delivery delay while minimizing risk.
- More reliable for emergency medicine delivery where safety and correctness are critical.

Although Greedy Best-First Search is faster, it may choose unsafe or expensive routes because it ignores actual movement costs. Therefore, **A*** provides the best balance between **real-time decision-making, optimality, and safety**, making it the most suitable algorithm for this scenario.

Summary Table

Aspect	Greedy Best-First Search	A* Search
Evaluation Function	h(n)	g(n) + h(n)
Uses Actual Cost	No	Yes
Uses Heuristic	Yes	Yes
Speed	Very Fast	Moderate
Optimal Path	No	Yes (with admissible heuristic)
Handles Dynamic Changes	Poor	Good
Suitable for Emergency Drone	Limited	Highly Suitable
Safety & Reliability	Lower	Higher

Final Conclusion:

For an AI-powered drone delivering emergency medicines in a dynamic flood-affected region, *A Search is the most suitable algorithm** because it balances actual travel cost with heuristic guidance, adapts better to changing conditions, and provides the safest and most optimal route.

Scenario 2. Smart City Traffic Signal Optimization (Hill Climbing & Simulated Annealing / Genetic Algorithm)

Problem Formulation

This is an **optimization problem**.

- **Objective:** Minimize
  - Total waiting time
  - Fuel consumption
  - Traffic congestion
- **Decision Variables:** Green signal duration, red signal duration, signal sequence, intersection coordination.
- **Constraints:**
  - Minimum and maximum signal timings.
  - Pedestrian crossing time.
  - Emergency vehicle priority.
  - Traffic regulations.

Why Traditional Search is Inefficient

- The number of possible signal combinations is enormous.
- Traffic conditions change continuously.
- Searching every possible solution is computationally expensive and too slow for real-time control.

---

i. Hill Climbing in this Scenario

Hill Climbing starts with an initial traffic signal plan and repeatedly moves to a neighboring solution that improves traffic flow.

Working:

1. Start with current signal timings.
2. Evaluate traffic performance.
3. Modify one or more signal timings.
4. If performance improves, accept the new solution.
5. Repeat until no better neighbor exists.

Limitations

- May stop at a local optimum.
- Cannot escape flat regions (plateaus).
- May fail on ridge-like landscapes.
- Does not guarantee the global optimum.

---

ii. Local Maxima, Plateaus, and Ridges

Local Maximum

- The algorithm finds a good traffic timing but not the best possible.
- Example: One intersection is optimized while city-wide traffic remains congested.

Plateau

- Many neighboring signal plans give the same waiting time.
- The algorithm cannot determine which direction to move.

Ridge

- Improvement requires changing several signals together.
- Hill Climbing changes only one step at a time and may fail to find the better solution.

---

iii. Simulated Annealing and Genetic Algorithm

Simulated Annealing

- Occasionally accepts worse solutions.

- Helps escape local maxima.
- Probability of accepting worse solutions decreases over time.

Advantages

- Escapes local optima.
- Better exploration.
- Produces near-optimal solutions.

Genetic Algorithm (GA)

Inspired by natural evolution.

Steps

1. Generate random signal timing plans.
2. Evaluate fitness.
3. Select the best plans.
4. Perform crossover.
5. Apply mutation.
6. Repeat until the best solution is found.

Advantages

- Searches many solutions simultaneously.
- Avoids local optima.
- Highly scalable.
- Adapts well to changing traffic.

---

iv. Recommendation

Recommended Approach: Genetic Algorithm

Justification

- Handles huge search spaces efficiently.
- Optimizes multiple objectives simultaneously.
- Escapes local optima.
- Easily adapts to changing traffic.
- Highly scalable for hundreds of intersections.

Conclusion: Genetic Algorithm is the most suitable approach because it provides high scalability, adaptability, and near-optimal traffic optimization.

Scenario 3. Autonomous Mars Rover (Online Search Agent)

i. Why Online Search Instead of Offline Search

Offline search assumes the complete environment is known before planning.
The Mars rover:

- Has no complete map.
- Discovers obstacles while moving.
- Receives new information continuously.

Therefore, **Online Search** is required.

---

ii. Challenges

Partial Observability

- Limited sensor range.
- Unknown terrain.
- Hidden obstacles.

Dynamic Environment

- Dust storms.
- Loose rocks.
- Sensor uncertainty.
- Newly discovered hazards.

These require continuous replanning.

---

iii. Internal Model

The rover maintains an internal map.
It continuously:

1. Collects sensor information.
2. Updates obstacle locations.
3. Marks explored areas.
4. Estimates safe routes.
5. Plans the next movement.

This map becomes more accurate as exploration continues.

---

iv. Online Search vs Other Search Methods

Feature	Uninformed Search	Informed Search	Online Search
Complete Map Needed	Yes	Yes	No
Uses Heuristics	No	Yes	Optional
Learns While Moving	No	No	Yes
Adapts to Changes	Poor	Moderate	Excellent
Suitable for Mars Rover	No	Limited	Yes

---

v. Recommendation

Recommended Approach: Online Search Agent

Justification

- Works without complete knowledge.
- Learns continuously.
- Updates routes dynamically.
- Safely avoids hazards.
- Ideal for uncertain environments.

Conclusion: Online Search provides the best balance of adaptability, uncertainty handling, and safe navigation.

Scenario 4. University Exam Timetable (Constraint Satisfaction Problem - CSP)
Problem Formulation
A CSP consists of:
Variables

- Exam time for each course.
- Exam hall assignment.
- Faculty assignment.

Domains
Possible values:

- Available dates.
- Available time slots.
- Available halls.

Constraints

1. No student has overlapping exams.
2. Hall capacity must not be exceeded.
3. Faculty must be available.
4. Subject ordering constraints.
5. Special accommodation requirements.

---

i. Variables, Domains, Constraints
Variables

- Course exam schedules.
- Hall allocation.
- Invigilator assignment.

Domains

- Morning/Afternoon sessions.
- Available halls.
- Available dates.

Constraints

- Student conflicts.
- Hall limits.
- Faculty availability.
- Subject prerequisites.
- Accessibility requirements.

---

ii. Backtracking Search
Steps:

1. Assign a time slot to one course.
2. Check constraints.
3. If valid, continue.
4. If conflict occurs, undo the assignment.
5. Try another time slot.
6. Continue until all exams are scheduled.

Backtracking systematically explores valid schedules.

---

iii. Constraint Propagation (Forward Checking)
Forward checking removes invalid future assignments immediately.
Example:

- If Mathematics is assigned Monday morning,
- That time slot is removed for all courses sharing students.

Advantages

- Reduces unnecessary search.
- Detects conflicts early.
- Improves efficiency.

---

iv. Real-world Challenges

- Thousands of students.
- Hundreds of courses.
- Limited halls.
- Faculty changes.
- Last-minute updates.

Recommended Strategy

Backtracking + Forward Checking + Constraint Propagation

Justification

- Reduces search space.
- Finds valid schedules faster.
- Scales efficiently for large universities.

Scenario 5. Strategic Game AI (Minimax & Alpha-Beta Pruning)

i. Minimax Algorithm

Minimax assumes:

- AI (MAX player) tries to maximize its score.
- Human player (MIN player) tries to minimize AI's score.

The algorithm explores future game states and chooses the move with the best guaranteed outcome.

---

ii. Computational Challenges

- Huge branching factor.
- Millions of possible moves.
- Deep search trees.
- High memory usage.
- Slow computation for real-time games.

Time complexity:

$O(b^d)$

where:

- **b** = branching factor
- **d** = search depth

---

iii. Alpha-Beta Pruning

Alpha-Beta pruning removes branches that cannot influence the final decision.

Alpha (α)

Best value MAX can guarantee.

Beta (β)

Best value MIN can guarantee.

If:

$\alpha \geq \beta$

the remaining branches are pruned because they cannot improve the result.

Advantages

- Reduces unnecessary node evaluation.
- Speeds up decision-making.
- Produces the same optimal move as Minimax.
- Enables deeper searches within time limits.

---

iv. Evaluation Function

The evaluation function estimates the quality of a game state.

Example factors:

- Number of units remaining.
- Health of units.
- Resources collected.
- Territory controlled.
- Defensive strength.
- Attack opportunities.
- Distance to objectives.

Example Formula:

Score = (Resources × 3) + (Army Strength × 5) + (Territory × 2) − (Enemy Strength × 4)

Higher scores indicate better positions for the AI.

---

v. Recommendation

Recommended Strategy: Alpha-Beta Pruning with Minimax
Justification

- Produces the same optimal decisions as Minimax.
- Significantly reduces computation.
- Handles very large game trees efficiently.
- Meets strict real-time decision-making requirements.
- Widely used in modern game AI.

Final Recommendations

Scenario	**Best Algorithm**	**Reason**
Smart Traffic Optimization	Genetic Algorithm	Scalable, adaptive, avoids local optima
Mars Rover Navigation	Online Search Agent	Handles unknown, dynamic environments
University Exam Timetabling	CSP with Backtracking + Forward Checking	Efficient constraint handling and scalability
Real-Time Strategy Game AI	Minimax + Alpha-Beta Pruning	Optimal decisions with faster computation